

CYK VARIANTS DESCRIPTION & ANALYSIS

Ángel Sánchez Paredes
ngsn0001

Abstract. We compare three algorithms for recognizing strings of a context-free grammar in Chomsky normal form: a naive recursive parser, a top-down parser with memoization, and the bottom-up Cocke–Younger–Kasami (CYK) algorithm. We analyze their expected running times and test them experimentally in Java, using both wall-clock time and an operation counter, on several grammars and input patterns

Keywords: context-free grammars, CYK algorithm, dynamic programming, parsing complexity

1 Introduction

Recognising whether a string belongs to the language of a context-free grammar is a basic problem in parsing and formal language theory. The CYK algorithm solves it in cubic time for grammars in Chomsky normal form, by dynamic programming over the substrings of the input. A naive recursive implementation of the same recurrence, without storing results, may instead take exponential time.

In this report we study three variants of this recurrence: the naive recursive algorithm, a top-down algorithm with memoization, and a bottom-up algorithm. We ask whether the cubic bound is visible in practice, whether either CYK variant is faster, and how the grammar and the input string influence the running time.

2 Preliminaries

2.1 Context-Free Grammars and Chomsky Normal Form

A context-free grammar (CFG) provides a set of structural rules for constructing or validating strings belonging to a formal language. For instance, in the case of balanced parentheses, a CFG allows nested structures to be defined recursively (a valid pair of parentheses may itself contain another valid structure).

Formally, a context-free grammar is defined as a 4-tuple $G = (V, \Sigma, R, S)$ [5], where:

- V is a finite set of nonterminal symbols (syntactic variables).
- Σ is a finite set of terminal symbols, disjoint from V ($V \cap \Sigma = \emptyset$), representing the actual characters of the alphabet.
- $R \subseteq V \times (V \cup \Sigma)^*$ is a finite set of production rules, where each rule maps a single nonterminal symbol $A \in V$ to a sequence of nonterminal and/or terminal symbols.
- $S \in V$ is the designated start symbol.

A string belongs to the language generated by G if and only if it can be derived from the start symbol through a finite sequence of production-rule applications.

To allow efficient processing by the CYK algorithm, the grammar must be expressed in Chomsky Normal Form (CNF) [1]. A grammar is in CNF if every production rule strictly follows one of the two forms below:

- $A \rightarrow B C$, where $A, B, C \in V$
- $A \rightarrow a$, where $a \in \Sigma$

2.2 The Cocke–Younger–Kasami Algorithm

The CYK algorithm [2,3,4] determines whether a string of length n belongs to $L(G)$ — the language generated by a grammar G — by examining every substring of the input. It relies on dynamic programming, evaluating whether a nonterminal symbol can generate a substring $s[i,j]$ by combining two smaller, previously solved substrings at some split point.

Formally, let $s[i, j]$ denote the substring of s from position i to $j - 1$. For a grammar in CNF, a nonterminal A derives $s[i, j]$ if either $j = i + 1$ and $A \rightarrow s[i]$ is a rule, or there are a rule $A \rightarrow B C$ and a split point k with $i < k < j$ such that B derives $s[i, k]$ and C derives $s[k, j]$. The string is accepted if and only if S derives $s[0, n]$. Each pair (A, i, j) is a subproblem, and the same subproblem is needed by many larger ones. Dynamic programming exploits this by solving each subproblem once and reusing the stored answer, either on demand (top-down with memoization) or in order of increasing substring length (bottom-up tabulation).

In this work, we evaluate three distinct variants for computing this dynamic-programming recurrence.

Example. Consider the grammar G_1 of Section 4.2 (balanced parentheses) and the input ‘ $(())$ ’. Fig. 1 shows for each substring the set of nonterminals that derive it. Length-one entries come from the terminal rules. The substring ‘ $()$ ’ at position 1 is derived by S via $S \rightarrow A B$, the substring ‘ $()$ ’ at position 1 by C via $C \rightarrow S B$, and the whole string by S via $S \rightarrow A C$ with $k = 1$. Since S derives $s[0, 4)$, the string is accepted.

Fig. 1. CYK table for ‘ $(())$ ’ under G_1 : nonterminals deriving the substring of length ℓ starting at position i ($\emptyset$ = none).

	$i = 0$ ‘ $($ ’	$i = 1$ ‘ $($ ’	$i = 2$ ‘ $)$ ’	$i = 3$ ‘ $)$ ’
$\ell = 1$	{A}	{A}	{B}	{B}
$\ell = 2$	$\emptyset$	{S}	$\emptyset$	
$\ell = 3$	$\emptyset$	{C}		
$\ell = 4$	{S}			

3 Algorithm Variants and Complexity Analysis

3.1 Naive Recursive Algorithm

This implementation evaluates the boolean function $\text{parseRec}(A, i, j, s)$ through pure recursion, without storing any previously computed subproblem. For a substring of length 1 ($j = i + 1$), it directly checks whether the rule $A \rightarrow s[i]$ exists in R . For substrings of length ≥ 2 , it iterates over the binary rules $A \rightarrow B C$ and over every possible split point, recursively invoking $\text{parseRec}(B, i, k, s)$ and $\text{parseRec}(C, k, j, s)$.

The main drawback of this approach is the severe redundancy in the computation: a specific subproblem (A, i, j) can be reached through multiple independent paths within the recursion tree. Since the algorithm recomputes these overlapping subproblems from scratch along every branch, the recursion tree grows exponentially, which makes the algorithm completely impractical for long input strings.

```
function parse(A, i, j, s):
    if j - i == 1:
        return A -> s[i] is a rule
    for each rule A -> B C:
        for k = i+1 to j-1:
            if parse(B, i, k, s) and parse(C, k, j, s):
                return true
    return false
```

3.2 Top-Down Variant with Memoization

To eliminate the redundant calls of the naive algorithm, this variant keeps the same recursive structure but introduces a memoization table $\text{table}[A][i][j]$. Each entry is initially set to an UNKNOWN (not-yet-computed) state. When parseRec is invoked, the algorithm first inspects the table: if the entry has already been evaluated, it directly returns the stored result. A byte value is used to encode the state of each entry: 0 = unknown, 1 = true, 2 = false.

The complexity of this approach is analyzed by bounding the total number of distinct subproblems and the cost per subproblem:

- Number of subproblems: there are $|V|$ possible nonterminals and $O(n^2)$ distinct index pairs (i, j) , resulting in a total of $O(|V| \cdot n^2)$ table entries.
- Work per subproblem: for each state (A, i, j) , the algorithm iterates over $O(n)$ split points k and checks the applicable binary rules $A \rightarrow B C$.

As a result, the total worst-case time complexity drops from exponential to cubic: $O(|V| \cdot |R| \cdot n^3)$, or $O(n^3)$ with respect to the input length n for a fixed grammar. The auxiliary space complexity is $O(|V| \cdot n^2)$ for storing the memoization table.

```
function parse(A, i, j, s):
    if table[A][i][j] is known:
        return table[A][i][j]
    if j - i == 1:
```

```

        result = (A -> s[i] is a rule)
    else:
        result = false
        for each rule A -> B C:
            for k = i+1 to j-1:
                if parse(B, i, k, s) and parse(C, k, j, s):
                    result = true; stop both loops
    table[A][i][j] = result
    return result

# accept iff parse(S, 0, n, s)

```

3.3 Bottom-Up Variant

This variant solves the subproblems iteratively, without recursion. It first fills the dynamic-programming table for all substrings of length $l = 1$, using the terminal rules $A \rightarrow a$. It then systematically computes the solutions for increasing substring lengths $l = 2, 3, \dots, n$. For a fixed length l and a starting index i , it iterates over every split point and every binary rule, checking whether both $\text{table}[B][i][k]$ and $\text{table}[C][k][j]$ hold true.

As with the top-down variant, this approach achieves an optimal worst-case time complexity of $O(|V| \cdot |R| \cdot n^3)$ and a space complexity of $O(|V| \cdot n^2)$. There is, however, a fundamental practical difference between the two: the bottom-up algorithm unconditionally fills the entire table for every possible subproblem up to length n , whereas the top-down variant works more lazily, computing only the states that are actually reachable from the start symbol S .

```

function parseBU(s):    # n = |s|
    for i = 0 to n-1:
        for each nonterminal A:
            table[A][i][i+1] = (A -> s[i] is a rule)
    for len = 2 to n:
        for i = 0 to n-len:
            j = i + len
            for each nonterminal A:
                table[A][i][j] = false
                for each rule A -> B C:
                    for k = i+1 to j-1:
                        if table[B][i][k] and table[C][k][j]:
                            table[A][i][j] = true; break both
    return table[S][0][n]

```

3.4 Comparative Discussion

Top-down and bottom-up have the same worst-case bound because both evaluate each of the $O(|V| \cdot n^2)$ subproblems (A, i, j) at most once, at $O(n \cdot |R|)$ cost each. The naive

algorithm has no table, so the same subproblem is recomputed along every path that reaches it. The two CYK variants differ mainly in how they visit the subproblems:

- **Lazy vs. strict evaluation.** Bottom-up always fills the whole table, while top-down only evaluates states reachable from S and can stop early, so it may be much faster on favourable inputs.
- **Cost per operation.** Bottom-up consists of plain nested loops over contiguous memory, whereas top-down pays for recursive calls and a memo-table check on every visit. We therefore expect bottom-up to have smaller constants when both do similar amounts of work.

Lower bound for the naive algorithm. Let the grammar contain $S \rightarrow S S$ and let $T(n)$ be the number of calls the naive algorithm makes on a string of length n in which every non-empty prefix is rejected (e.g. one starting with a symbol that no rule can produce first). For each split point k , the left call $\text{parseRec}(S, 0, k)$ then returns false, so the loop is never cut short and all $k = 1, \dots, n-1$ are tried. Hence

$$T(n) \geq 1 + T(1) + T(2) + \dots + T(n-1), \quad T(1) = 1.$$

We claim $T(n) \geq 2^{n-1}$. This holds for $n = 1$, and if it holds for all $k < n$, then $T(n) \geq 1 + (2^0 + 2^1 + \dots + 2^{n-2}) = 2^{n-1}$. So the naive algorithm needs $\Omega(2^n)$ time in the worst case, and each added character doubles the work (a factor of 4 for two characters, as observed in Section 5.1). Note that this is the worst case only: if a derivation is found on the first branch, the naive algorithm can finish in linear time.

Dependence on grammar and input. All three algorithms stop searching as soon as one split point succeeds and evaluate the conjunction lazily, so their running time depends on the input and not only on n . Rejected strings force an exhaustive search over all split points, while accepted ones may finish early. The position of an error matters for the recursive variants: if the first call fails immediately (e.g. a symbol that no rule can start with, at the beginning of the string), the second call is never made, whereas an error at the end is only found after many successful subderivations, followed by backtracking. The grammar matters as well: if the first component of a rule can only derive very short strings (as $A \rightarrow a$ in G_2), the recursive variants can prune almost everything. Bottom-up visits every table entry regardless, so only its early exit on success changes its operation count, not its asymptotic cost. The experiments in Section 5 are designed around these differences.

4 Experimental Setup

4.1 Measurement Methodology

To empirically validate the theoretical estimates, performance measurements are carried out in a controlled setting:

- Repetitions and outlier filtering: each test string is evaluated over 10 independent runs. To reduce system noise (such as operating-system context switches or JIT compilation effects), the minimum and maximum recorded times are discarded, and the arithmetic mean of the remaining 8 measurements is computed.

- JVM warm-up and garbage collection: before recording the measurements, a warm-up run is performed with the longest test string, to force the Java JIT compiler's optimizations ahead of time. In addition, `System.gc()` is invoked explicitly before each measurement cycle.
- Operation counter (opCounter): besides the wall-clock execution time, an internal step counter (opCounter) is maintained by each parser. In the naive and top-down variants, opCounter counts the number of recursive calls performed; in the bottom-up variant, it counts the iterations of the innermost split-point loop. Since these two metrics count different primitive operations, they are useful for measuring the asymptotic growth rate of each algorithm individually, but not for making a direct absolute-speed comparison between variants.

4.2 Grammars and Input Patterns Used

We use three grammars in CNF, over the alphabets indicated:

- **G₁ (balanced parentheses):** $S \rightarrow S S \mid A B \mid A C$, $C \rightarrow S B$, $A \rightarrow '(', B \rightarrow ')'$.
- **G₂ (strings over {a, b} starting with a):** $S \rightarrow A B$, $A \rightarrow a$, $B \rightarrow B B \mid a \mid b$.
- **G₃ (dense grammar):** the “stupid” grammar from Step 1, whose rules never derive a terminal string, so every input is rejected and no shortcuts exist.

The input patterns for G₁ are $()() \dots ()$ (accepted), $((\dots))$ (nested, accepted), $)()() \dots ()$ (invalid symbol first), and $()() \dots ()($ (invalid symbol last). For G₂ we use strings starting with a (accepted) and strings starting with b (rejected). For G₃ we use $s \dots s$. In all cases n denotes the string length.

5 Experimental Results

5.1 Basic Experiments: Confirming Theoretical Growth

Hypothesis. Following the analysis in Section 3, we expect the naive algorithm to grow exponentially on inputs that force it to explore its whole recursion tree, and both memoized variants to stay within $O(n^3)$. Experiment 1 tests the first part on the balanced parentheses grammar G₁, using one accepted and one rejected string.

Table 1. Experiment 1: growth of the three variants on $()() \dots ()$ (balanced parentheses grammar).

n	Naive		Top-down		Bottom-up	
	Time (ns)	Ops	Time (ns)	Ops	Time (ns)	Ops
2	38312	6	23000	6	27962	4
4	34587	14	24000	14	49425	33
6	61937	22	46350	22	48787	113
8	34412	30	24187	30	56525	270
10	26287	38	30812	38	87725	530

n	Naive		Top-down		Bottom-up	
	Time (ns)	Ops	Time (ns)	Ops	Time (ns)	Ops
12	29337	46	21375	46	92662	919
14	34100	54	31550	54	105712	1463
16	18662	62	19650	62	105650	2188

Experiment 1A: accepted string $()() \dots ()$ (Table 1). Contrary to our initial expectation, the naive algorithm does not blow up here: it performs $4n - 2$ operations, exactly as many as top-down. The string is derived by $S \rightarrow S S$ already at the first split point $k = 2$, and since the search over split points stops at the first success, it returns true along this branch and never asks for the same subproblem (A, i, j) twice. Memoization therefore brings no advantage in this case. Bottom-up, in contrast, fills the table regardless, and its operation count grows superlinearly (2188 operations at $n = 16$, against 62 for the other two).

Table 2. Experiment 1: growth of the three variants on $()() \dots ()$ (rejected string: extra closing parenthesis first).

n	Naive		Top-down		Bottom-up	
	Time (ns)	Ops	Time (ns)	Ops	Time (ns)	Ops
3	24450	10	21175	10	36025	16
5	14062	46	21575	31	60037	73
7	10275	190	21187	64	89412	197
9	13900	766	16075	109	80325	414
11	24337	3070	20812	166	106712	750
13	59425	12286	20012	235	119262	1231

Experiment 1B: rejected string $)() \dots ()$. Without a valid derivation the naive algorithm must exhaust the recursion tree. Its operation count grows by a factor of ≈ 4 for every two added characters ($10 \rightarrow 46 \rightarrow 190 \rightarrow 766 \rightarrow 3070 \rightarrow 12286$), i.e. as $4^{(n/2)} = 2^n$, which matches the exponential expectation. Top-down avoids this through memoization and needs exactly $1.5n^2 - 1.5n + 1$ operations, i.e. quadratic. We also reproduced these counts by hand-tracing small cases to rule out an implementation error.

5.2 Highlighting Differences Between Variants

Hypothesis. Both CYK variants are cubic in the worst case, but two effects should separate them. Top-down only visits subproblems reachable from S and can stop early,

both when one of the splits succeeds and when the first half of a rule application fails, whereas bottom-up always fills the whole table. On the other hand, bottom-up avoids the overhead of recursion and memo-table look-ups, so its cost per operation should be lower. Experiment 2 probes both effects on G_1 .

Table 3. Experiment 2: top-down vs. bottom-up on $((...))$ (balanced parentheses grammar).

n	Top-down			Bottom-up		
	Time (ns)	Ops	Time/n ³	Time (ns)	Ops	Time/n ³
100	1052175	335852	1.052	1153787	661651	1.154
200	5332937	2676702	0.667	5911050	5313301	0.739
300	17692062	9022552	0.655	12737312	17954951	0.472
400	41870225	21373402	0.654	27892225	42586601	0.436

Experiment 2A: nested string $((...))$ (Table 3). The operation counts stabilise at $\approx n^3/3$ for top-down and $\approx 2n^3/3$ for bottom-up (21.4M and 42.6M for $n = 400$), confirming cubic growth for both. Nevertheless, bottom-up is faster in wall-clock time at $n = 400$ (27.9 ms against 41.9 ms). This corresponds to ≈ 1.96 ns per counted operation for top-down and ≈ 0.65 ns for bottom-up. The two counters measure different things (recursive calls versus split-loop iterations), so this ratio is only indicative, but it is consistent with recursion and table look-ups being considerably more expensive than plain contiguous loops.

Table 4. Experiment 2: top-down vs. bottom-up on $()()...()$.

n	Top-down			Bottom-up		
	Time (ns)	Ops	Time/n ³	Time (ns)	Ops	Time/n ³
100	109500	398	0.1095	952837	540425	0.953
200	71287	798	0.0089	4172987	4328350	0.522
300	104012	1198	0.0039	5726900	14613775	0.212
400	124637	1598	0.0019	12423900	34646700	0.194

Table 5. Experiment 2: top-down vs. bottom-up on $()()...()()$ (extra opening parenthesis last).

n	Top-down			Bottom-up		
	Time (ns)	Ops	Time/n ³	Time (ns)	Ops	Time/n ³
101	1196637	450326	1.161	1133787	560625	1.100
201	8255750	3550651	1.017	2405875	4408750	0.296
301	28541237	11925976	10.47	6073162	14794375	0.223
401	64976425	28201301	1.008	12611325	34967500	0.196

Table 6. Experiment 2: top-down vs. bottom-up on $)() \dots ()$ (extra closing parenthesis first).

n	Top-down			Bottom-up		
	Time (ns)	Ops	Time/n ³	Time (ns)	Ops	Time/n ³
101	12626	15151	0.0123	1024312	560625	0.994
201	217162	60301	0.0267	5600862	4408750	0.690
301	407087	135451	0.0149	5642600	14794375	0.207
401	709150	240601	0.0110	12689500	34967500	0.197

Experiment 2B: repetitive strings (Tables 4–6). On the accepted string $()() \dots ()$ (Table 4), top-down needs only $4n - 2$ operations (1598 and 0.12 ms for $n = 400$), about 100 times faster than bottom-up (34.6M operations, 12.4 ms), which cannot exploit reachability and still does cubic work. When the string is rejected, the position of the invalid symbol is decisive:

- Invalid symbol first, $)() \dots ()$ (Table 6): no rule derivable from S can start with a closing parenthesis, so the conjunction of the two halves fails at its first operand and the search is cut off immediately. Top-down stays quadratic ($\approx 1.5n^2$, 240 601 operations for $n = 401$).
- Invalid symbol last, $()() \dots ()$ (Table 5): top-down successfully validates almost the entire string and only fails at the last symbol, after which it must backtrack over many earlier splits. Its cost becomes cubic ($\approx 0.44n^3$, 28.2M operations for $n = 401$), and bottom-up is then faster in wall-clock time (12.6 ms against 65.0 ms).
- The Time/n³ columns support these observations. For top-down on $((\dots))$ and $()() \dots ()$ the ratio stabilises (≈ 0.65 and ≈ 1.0 ns), consistent with cubic growth, whereas on $()() \dots ()$ and $)() \dots ()$ it approaches zero, since the running time is sub-cubic. For bottom-up the ratio decreases with n , reaching ≈ 0.2 ns in Tables 4–6 while still falling in Table 3. The larger values for small n probably reflect fixed overheads such as JIT warm-up.

5.3 Other Grammars

Experiment 3 uses the grammar G_2 (strings starting with a), whose structure restricts how subproblems can be combined; Experiment 4 uses the grammar G_3 of Table 9, which offers no such restrictions.

Table 7. Experiment 3: strings that start with a, input belonging to the language.

n	Result	Top-down		Bottom-up	
		Time (ns)	Ops	Time (ns)	Ops
100	Yes	89150	199	236587	9900
200	Yes	46812	399	396487	39800
300	Yes	70537	599	907250	89700
400	Yes	74312	799	1241675	159600

Table 8. Experiment 3: strings that start with a, input not belonging to the language.

n	Result	Top-down		Bottom-up	
		Time (ns)	Ops	Time (ns)	Ops
100	No	75737	100	559375	14751
200	No	66900	200	637337	59501
300	No	32187	300	1255275	134251
400	No	107725	400	2076487	239001

Experiment 3: G_2 (Tables 7 and 8). Neither variant reaches cubic behaviour. On accepted strings top-down is linear ($2n - 1$ operations) and bottom-up quadratic ($n(n - 1)$); on rejected strings top-down needs exactly n operations and bottom-up $\approx 1.5n^2$. Hence $O(n^3)$ is only an upper bound: when the grammar makes the first operand of a rule fail or succeed early, the loop over split points is left before all $O(n)$ cut points are tried.

Table 9. Experiment 4: the “stupid” grammar on s...s.

n	Result	Top-down		Bottom-up	
		Time (ns)	Ops	Time (ns)	Ops
100		581737	338251	57005	333300
200		3902600	2686501	3168725	2666600
300		12403100	9044751	4738200	8999900
400		28600675	21413001	10739837	21333200

Experiment 4: G_3 (Table 9). This grammar is the worst case for both variants: there are no shortcuts, so both perform $\approx n^3/3$ operations (21.4M and 21.3M for $n = 400$) and must complete the whole table. Here bottom-up is 2.7 times faster (10.7 ms against 28.6 ms), which shows the price of recursion when no pruning is possible.

6 Conclusion

The experiments support the theoretical analysis, and show that the grammar and the input string matter as much as the asymptotic bound:

- **Naïve.** Exponential growth appears when the algorithm has to exhaust its recursion tree, e.g. on rejected strings. If a derivation is found along the first branch, it runs in linear time.
- **Top-down.** It is cubic in the worst case, but lazy evaluation makes it linear or quadratic on inputs with early success or early failure. Errors late in the string are its bad case.
- **Bottom-up.** It cannot prune, but its cost per operation is lower, so it is up to 2.7 times faster when the whole table has to be computed (nested strings, G_3), presumably because of the absence of recursion overhead.

No variant is best everywhere. Operation counts and wall-clock times complement each other: the former reveal the growth rate, the latter the constant factors.

References

1. Chomsky, N.: On certain formal properties of grammars. *Information and Control* 2(2), 137–167 (1959)
2. Kasami, T.: An efficient recognition and syntax-analysis algorithm for context-free languages. Scientific Report AFCRL-65-758, Air Force Cambridge Research Laboratory, Bedford, MA (1965)
3. Younger, D.H.: Recognition and parsing of context-free languages in time n^3 . *Information and Control* 10(2), 189–208 (1967)
4. Cocke, J., Schwartz, J.T.: Programming languages and their compilers: Preliminary notes. Technical report, Courant Institute of Mathematical Sciences, New York University (1970)
5. Hopcroft, J.E., Ullman, J.D.: Introduction to Automata Theory, Languages, and Computation. Addison-Wesley (1979)